fit@hcmus
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG - HCM
KHOA CÔNG NGHỆ THÔNG TIN

Seminar — Đề tài 9

ANSIBLE CONFIGURATION MANAGEMENT

Agentless · Idempotent · Infrastructure as Code

Giảng viên hướng dẫn: Lê Hà Minh
Sinh viên thực hiện:
Lê Quốc Hưng, Nguyễn Đức Duy, Nguyễn Thành Dự, Âu Bảo Trân, Mai Thúc Hải Đăng

01 Bài toán & sự ra đời

02 Kiến trúc Agentless

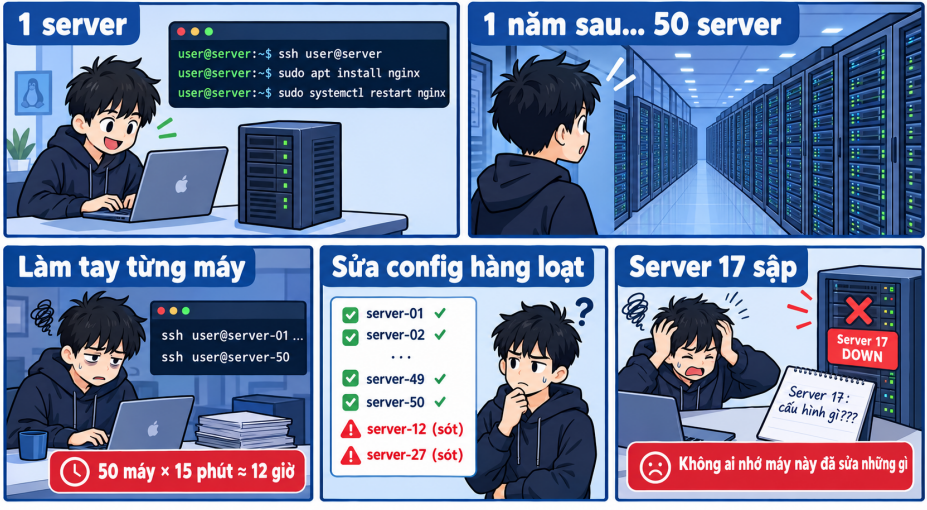
03 Các thành phần cơ bản

04 Idempotence

05 Module chuyên dụng vs Shell

06 Ansible Vault

Bài toán: từ 1 server lên 50 server



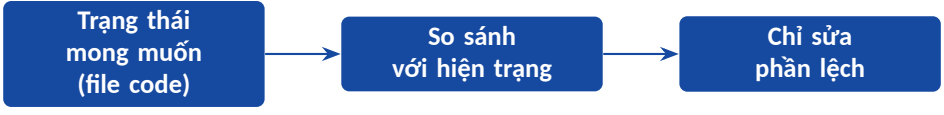
Thử giải bằng tay và bằng script bash

	Làm tay	Script bash
Nhiều máy	Rất tốn công, dễ sót	Tự viết vòng lặp, tự xử lý lỗi
Chạy lại	Gõ lại từ đầu	Lỗi ở bước đã làm rồi (`mkdir: File exists`, `useradd: already exists`)
Biết máy đang ra sao	Không	Không
Tài liệu hoá	Nằm trong đầu người làm	Script dài, khó đọc

> **Hậu quả — Configuration drift:** mỗi máy lệch đi một kiểu sau nhiều lần sửa tay.

Hướng giải: Configuration Management & IaC

- Không viết *"làm thế nào"* mà **mô tả trạng thái mong muốn**: *"nginx phải được cài, config phải có nội dung X, service phải đang chạy"*.
- Công cụ tự so sánh với hiện trạng và **chỉ sửa phần lệch** → chạy lại bao nhiêu lần cũng an toàn (**idempotent**).
- **Infrastructure as Code**: cấu hình là file code → lưu Git, review, dựng lại server từ file.



Thế hệ công cụ đầu tiên

Công cụ	Năm	Agent	Mô hình	Ngôn ngữ
CFEngine	1993	Có agent	Pull	DSL riêng
puppet	2005	Có agent	Pull	Puppet DSL
CHEF	2009	Có agent (Chef Infra Client)	Pull	Ruby DSL
SALTSTACK	2011	`salt-minion` (Salt SSH thì agentless)	Push / lịch chạy	YAML + Jinja2

Giải quyết được bài toán, nhưng phải:

- Cài và cập nhật **agent trên từng máy**, dựng **server trung tâm**, quản lý **chứng chỉ/key**.
- Học **DSL hoặc Ruby** trước khi viết được cấu hình đầu tiên.

Ansible ra đời (2012)

Michael DeHaan



Tác giả Cobbler, đồng tác giả Func tại Red Hat

2012 — Phát hành Ansible

2015 — Red Hat mua lại

ANSIBLE

Ý tưởng

Tận dụng thứ máy Linux nào cũng có sẵn: SSH và Python.

Linux + SSH + Python

1. Agentless

- Không cài agent
- Không cần server trung tâm



2. YAML dễ đọc

- Đọc như tài liệu
- Bắt đầu trong vài phút

```
- name: Cài nginx
  hosts: webservers
  tasks:
    - apt: name=nginx
      state=present
```

3. Push

- Chạy khi bạn ra lệnh (có ansible-pull nếu cần)

```
$ ansible all -m ping
web1 | SUCCESS
web2 | SUCCESS
```



4. Tên gọi

- Thiết bị liên lạc tức thời giữa các hành tinh trong truyện của Ursula K. Le Guin



01 Bài toán & sự ra đời

02 Kiến trúc Agentless

03 Các thành phần cơ bản

04 Idempotence

05 Module chuyên dụng vs Shell

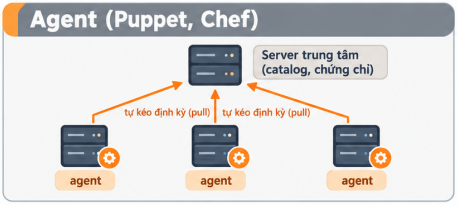
06 Ansible Vault

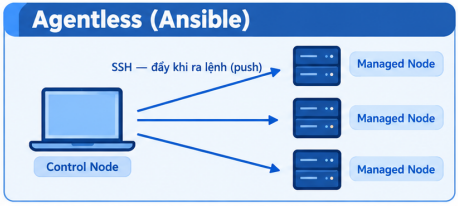
Agentless là gì?

- **Agent** = chương trình chạy thường trực trên mỗi máy được quản lý, liên lạc với server trung tâm để nhận cấu hình (Puppet agent, Chef Infra Client, salt-minion).
- **Agentless** = không cài agent riêng của Ansible trên máy đích. Ansible chỉ cài trên **Control Node**, khi cần thì kết nối tới **Managed Node** qua kênh có sẵn (**SSH** với Linux, WinRM/SSH với Windows), làm xong thì rút ra.

> Ansible **chủ động đẩy** (push) cấu hình khi được ra lệnh, thay vì mỗi máy phải cài agent và tự kéo (pull) định kỳ.

Agent vs Agentless





Tiêu chí	Agent	Agentless
Cài trên máy đích	Agent, phải cập nhật	Không — dùng SSH + Python có sẵn
Hạ tầng phụ	Server trung tâm, chứng chỉ	Chỉ cần Control Node
Ai mở kết nối	Agent tự kéo (pull)	Control Node đẩy (push)
Tài nguyên máy đích	Agent chạy nền thường trực	Chỉ tốn khi đang chạy task
Tự sửa drift	Có (chạy định kỳ)	Khi chạy lại playbook / lập lịch

Hai tiêu chí độc lập

	Push	Pull
Agentless	Ansible	ansible-pull
Agent	Salt	Puppet, Chef

Yêu cầu của Control Node và Managed Node

	Control Node	Managed Node (Linux)
Ansible	Cài `ansible` / `ansible-core`	Không cần
Python	Phiên bản được bản Ansible hỗ trợ	Cần cho phần lớn module (không bắt buộc cùng phiên bản với Control Node)
SSH	SSH client	SSH server đang chạy, mở cổng TCP 22
Khác	Inventory, playbook, private key	Tài khoản có shell; quyền `sudo` nếu task cần root

- Control Node phải là **Linux/macOS** (Windows thì dùng WSL); Managed Node có thể là Windows.
- Module `raw` chạy được cả khi máy đích chưa có Python → dùng để cài Python lần đầu.
- `become: true` = chạy bằng `sudo`; **không tự cấp quyền** sudo (`-K` để nhập mật khẩu sudo).

01 Bài toán & sự ra đời

02 Kiến trúc Agentless

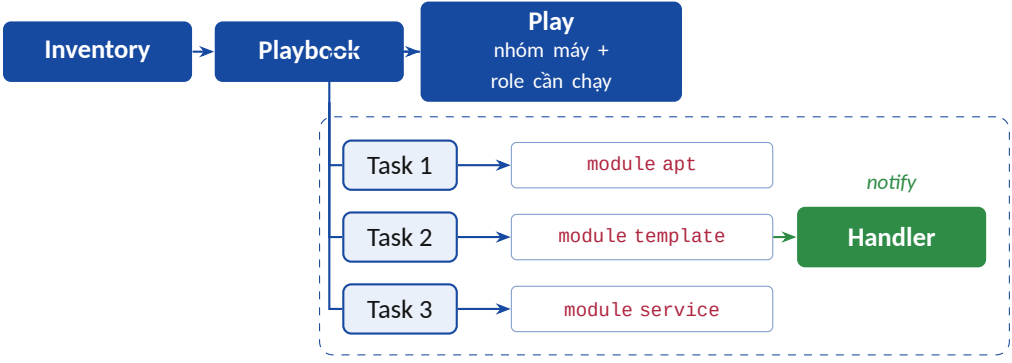
03 Các thành phần cơ bản

04 Idempotence

05 Module chuyên dụng vs Shell

06 Ansible Vault

Bức tranh tổng thể



Role = đóng gói tasks / handlers / templates / vars thành thư mục chuẩn

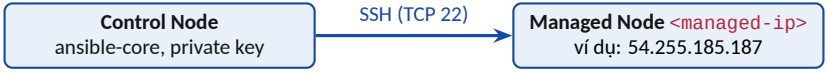
Inventory

```
# inventory
[webservers]
node1 ansible_host=54.255.185.187

[webservers:vars]
ansible_user=ubuntu
```

- Inventory = **danh sách máy** Ansible quản lý, chia thành **nhóm**.
- Nhóm mặc định `all`; biến tách ra `group_vars/` và `host_vars/`.
- **Cụ thể hơn thắng**: `host_vars` > `group_vars/<nhóm>` > `group_vars/all`.

Kết nối SSH key




```
ssh-keygen -t ed25519 -f ~/.ssh/ansible_lab
ssh-copy-id -i ~/.ssh/ansible_lab.pub ubuntu@54.255.185.187
ansible webservers -i inventory -m ansible.builtin.ping
# "pong" = SSH + Python OK (không phải ICMP)
```

```
# inventory
[webservers]
node1 ansible_host=54.255.185.187

[webservers:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/ansible_lab
```

- Private key **không bao giờ rời control node**; public key nằm trong `authorized_keys` của máy đích.

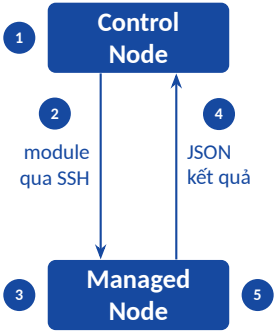
Playbook & Task

```yaml
- name: Cài nginx
  hosts: webservers
  become: true
  tasks:
    - name: Đảm bảo nginx đã cài
      ansible.builtin.apt:
        name: nginx
        state: present
```

- **Play**: `hosts`, `become`, `vars`, `tasks` (chạy từ trên xuống), `handlers`.
- **Task** = `name` + module + tham số; hay kèm `when`, `loop`, `register`, `notify`.
- `ansible.builtin.apt` = tên đầy đủ **(FQCN)**: collection `ansible.builtin` + module `apt`.
- **YAML**: thụt lề bằng **space**, `key: value`, list dùng `-`.
- **Ad-hoc** (`ansible webservers -i inventory -m ping`): chạy **1 module, 1 lần** từ dòng lệnh, không cần file. **Playbook**: nhiều task, lưu Git, chạy lại được.

Một task chạy như thế nào?

1. Control node đọc inventory, playbook, biến → đóng gói module + tham số.
2. SSH vào managed node, tạo thư mục tạm, chuyển mã module sang.
3. Managed node chạy module bằng Python (dùng `become` nếu cần quyền root).
4. Module trả **JSON** → Ansible báo `ok` / `changed` / `failed`.
5. Xoá file tạm; kết quả task (gói, file cấu hình) vẫn giữ.



Role

```
roles/nginx/
├── defaults/main.yml       # biến mặc định
├── handlers/main.yml       # handler restart Nginx
├── tasks/main.yml          # các task triển khai
├── templates/nginx.conf.j2 # template cấu hình
└── vars/vault.yml          # secret đã mã hoá
```

- Role **không tự chạy**; `site.yml` gọi bằng `roles: [nginx]`.
- File Vault được nạp qua `vars_files` trước khi role chạy.
- Ưu tiên biến: `defaults` < `group_vars` < `host_vars` < `vars` (role) < `-e` (extra vars **luôn thắng**).

Template Jinja2

- `{{ biến }}` in giá trị · `{% if %}` / `{% for %}` điều khiển · `{# #}` comment.
- Filter hay dùng: `default(80)`, `join(' ')`.
- Render **trên control node** rồi mới đẩy file sang máy đích.

	`copy`	`template`
Nội dung	Nguyên văn	Render biến rồi đẩy
Dùng cho	File tĩnh	File đổi theo host/môi trường

```
server {
    listen {{ http_port }} default_server;
    server_name {{ server_name }};
    root {{ document_root }};
    location / {
        try_files $uri $uri/ =404;
    }
}
```

Handler

- Chỉ chạy khi task `notify` báo **changed** — config không đổi thì không restart.
- Chạy **cuối play**, **1 lần mỗi host** dù được notify nhiều lần.
- Tên trong `notify` phải **khớp** tên handler.

> Đề yêu cầu *restart*; thực tế nên dùng `state: reloaded` (không downtime) khi chỉ đổi config.

```yaml
- name: Render nginx.conf
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: >-
      /etc/nginx/sites-available/default
  notify: Restart Nginx

# handlers/main.yml
- name: Restart Nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

01 Bài toán & sự ra đời

02 Kiến trúc Agentless

03 Các thành phần cơ bản

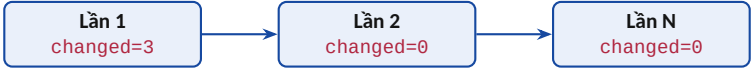
04 Idempotence

05 Module chuyên dụng vs Shell

06 Ansible Vault

Idempotent là gì?

- **Idempotent** (tính luỹ đẳng): một thao tác áp dụng **nhiều lần** cho **cùng kết quả** như áp dụng **một lần**.
- Trong Ansible: chạy playbook **1 lần hay 100 lần** → máy luôn ở **cùng trạng thái mong muốn**, không tác dụng phụ. Lần đầu sửa những gì còn lệch (`changed`), các lần sau chỉ báo `ok`.
- Ví dụ đời thường: bấm **nút tầng 5** thang máy nhiều lần vẫn chỉ đi tầng 5; còn **công tắc đèn** bấm 2 lần thì về lại trạng thái cũ → không idempotent.



> Vì sao là **cốt lõi**: chạy lại playbook luôn **an toàn** — sửa được máy bị lệch cấu hình, chạy tiếp sau khi lỗi giữa chừng, không lo cài trùng hay ghi đè sai.

Khai báo vs Mệnh lệnh

- Shell script: *"làm thế nào"*.
  Ansible: *"kết quả cuối là gì"*.
- Module: **kiểm tra hiện trạng** → khớp thì `ok`, lệch thì sửa vừa đủ → `changed`.

PLAY RECAP	Ý nghĩa
`ok`	Thành công (bao gồm cả `changed`)
`changed`	Đã thay đổi máy đích
`failed`	Lỗi, dừng play trên host đó
`unreachable`	Không SSH được tới host
`skipped`	`when` trả về false

Xem trước: `--check` & `--diff`

- `--check`: **mô phỏng**, không đổi gì trên máy đích.
- `--diff`: hiện **dòng thêm/bớt** trong file.
- Giới hạn: `shell`/`command` bị **skip** trong check mode (trừ khi có `creates`/`removes`).

> `ansible-playbook site.yml --check --diff` — xem trước thay đổi trước khi áp dụng thật.

01 Bài toán & sự ra đời

02 Kiến trúc Agentless

03 Các thành phần cơ bản

04 Idempotence

05 Module chuyên dụng vs Shell

06 Ansible Vault

command vs shell

	command	shell
Chạy qua `/bin/sh`	Không	Có
Pipe, redirect, `$VAR`	Không	Có
Rủi ro shell injection	**Thấp**	**Cao**

- **Bẫy**: `command: echo data >> /tmp/x` → **không tạo file** (`>>` thành đối số) nhưng vẫn báo `changed`.
- `raw` (không cần Python, dùng bootstrap) và `script` (chạy script có sẵn) chỉ dùng trường hợp đặc biệt.

Vì sao module chuyên dụng tốt hơn?

- `shell`/`command` là **“hộp đen”**: Ansible chỉ thấy `rc`, `stdout` → **luôn báo changed**.
- Module chuyên dụng **kiểm tra trước khi làm**:
  - `copy`: so checksum, giống thì `ok`.
  - `lineinfile`: dòng đã có thì `ok`, chưa có thì thêm.
  - `systemd_service`: `state: started` chỉ khởi động khi đang dừng.
- Kèm theo: hỗ trợ `--check`/`--diff`, trả dữ liệu có cấu trúc, an toàn hơn, dễ đọc.

Khi buộc phải dùng shell

```
- ansible.builtin.shell: tar -xzf app.tar.gz -C /opt/app
  args:
    creates: /opt/app/README     # đã có file này thì bỏ qua

- ansible.builtin.command: uname -r
  changed_when: false            # lệnh chỉ đọc
```

- `creates` / `removes`: cho Ansible biết khi nào lệnh **đã làm rồi** → bỏ qua, giữ idempotent.
- `changed_when: false`: lệnh chỉ đọc thì **không báo changed**.

01 Bài toán & sự ra đời

02 Kiến trúc Agentless

03 Các thành phần cơ bản

04 Idempotence

05 Module chuyên dụng vs Shell

06 Ansible Vault

Vault là gì?

- Secret để **plain text** trong Git sẽ **lộ vĩnh viễn** (Git nhớ lịch sử).
- Vault mã hoá **AES256** → secret vẫn nằm trong Git nhưng đã mã hoá.
- Lệnh: `create`, `encrypt`, `view`, `edit`, `decrypt`, `rekey`; `encrypt_string` mã hoá **từng giá trị** (tên biến vẫn đọc được).
- Chạy: `--ask-vault-pass` (chạy tay) hoặc `--vault-password-file` (tự động; file `chmod 600` + `.gitignore`).

Cách tổ chức

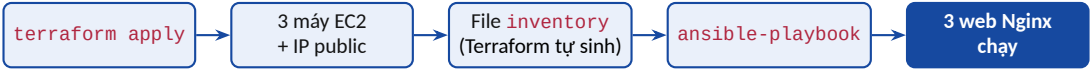
```
group_vars/web/
├── vars.yml    # công khai:  app_secret: "{{ vault_app_secret }}"
└── vault.yml   # ĐÃ MÃ HOÁ:  vault_app_secret: "..."
```

- Tiền tố `vault_` → grep được tên biến, biết ngay giá trị thật nằm trong vault.
- Task dùng secret thêm `no_log: true` để **không in secret** ra màn hình.

Hạn chế của Ansible

- **Chậm khi nhiều máy**: mỗi task phải SSH vào và đẩy module sang; mặc định chỉ chạy song song `forks = 5` máy.
- **Không tự sửa khi lệch cấu hình**: không có agent nên ai sửa tay trên máy thì Ansible không biết → phải chạy lại playbook (hoặc lên lịch chạy định kỳ).
- **Playbook lớn khó đọc**: YAML + Jinja2 + `when`/`loop` dễ rối, debug khó hơn code thông thường.
- **Không quản lý vòng đời hạ tầng**: không lưu state như Terraform → không hợp để tạo/xoá VM hàng loạt.

Ví dụ 1: Terraform dựng máy, Ansible cấu hình

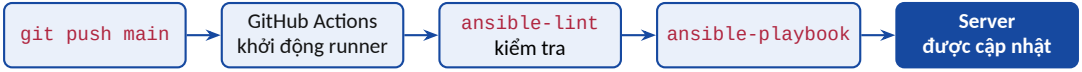



```
# main.tf
resource "aws_instance" "web" {
  count         = 3
  ami           = "ami-xxxx"      # Ubuntu
  instance_type = "t3.micro"
}
resource "local_file" "inventory" {
  filename = "inventory"
  content  = join("\n", concat(["[webservers]"],
               aws_instance.web[*].public_ip))
}
```

```
terraform apply -auto-approve
ansible-playbook -i inventory site.yml
```

- Terraform lo **“có máy”**: tạo VM, mạng, security group; nhớ state để sửa/xoá.
- Ansible lo **“máy chạy đúng”**: cài gói, render config, bật service.
- Không ai ghi IP bằng tay; cần 10 máy thì sửa `count = 10`.

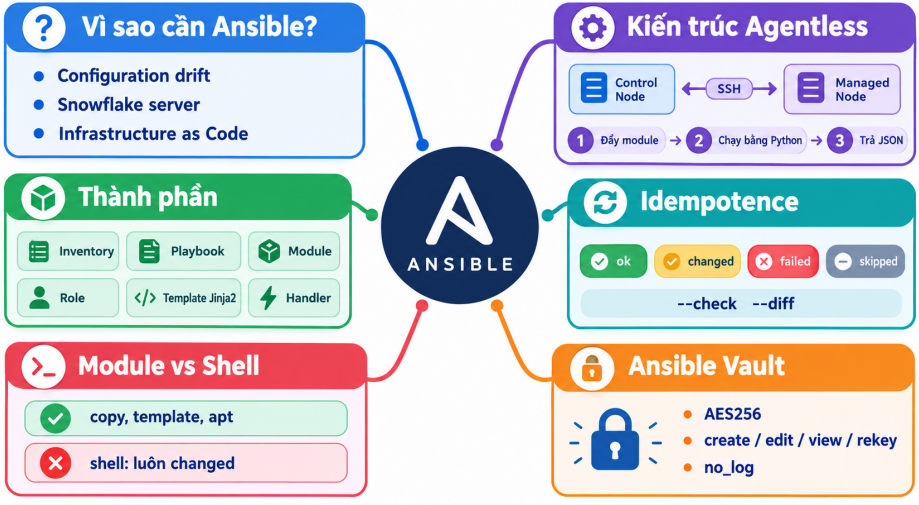
Ví dụ 2: CI/CD — push code là chạy playbook




```yaml
# .github/workflows/deploy.yml
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    env:
      ANSIBLE_HOST_KEY_CHECKING: "False"
    steps:
      - uses: actions/checkout@v4
      - run: pip install ansible ansible-lint
      - run: ansible-lint site.yml
      - run: |
          echo "${{ secrets.SSH_KEY }}" > key && chmod 600 key
          echo "${{ secrets.VAULT_PASS }}" > .vault_pass
          ansible-playbook -i inventory site.yml \
            --private-key key --vault-password-file .vault_pass
```

- Private key và mật khẩu Vault cất trong **GitHub Secrets**, không nằm trong repo.
- `ansible-lint` lỗi → pipeline dừng, không đụng tới server.
- Nhờ **idempotent**: push 10 lần thì chỉ phần thay đổi mới báo `changed`.

Mindmap kiến thức Ansible



fit@hcmus
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG - HCM
KHOA CÔNG NGHỆ THÔNG TIN

Cảm ơn thầy và các bạn đã lắng nghe!